UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

PROJECTE DE TECNOLOGIES DE LA INFORMACIÓ (PTI)

Informe de pràctiques

Blockchain

Autor Gabriel Calvo Catalán, Lucas Díaz López
Titulació Grau en Enginyeria Informàtica
Especialitat Tecnologies de la Informació
Curs 2026-2027, quadrimestre de tardor
Data de lliurament Octubre de 2026

1. Informació sobre l'ús d'IA

1) Com has aplicat la IA?
Ha ajudat en millorar la aparença i la redacció del informe i debugar codi.

2) Quines eines has fet servir?
Claude.

3) Quin ordre has fet servir?
Primer hem escrit el codi i despres li hem passat al no funcionar..

4) Mètode per al codi
Chat

5) Has fet servir agents?
No.

6) Temps estalviat
15%

7) Millora de la qualitat
Molt alta en l'informe

8) Impacte en l'aprenentatge
Positiu, ja que ens ha ajudat a trobar errors.

9) Ús recomanat
Fer-la servir com a ultim recurs si no ets capaç de debugar el codi,

2. Explicació tècnica

2.1 Entorn

La pràctica s'ha fet en un equip amb Ubuntu i Python 3. S'ha creat l'entorn virtual prbc, s'hi han instal·lat Flask, requests i Werkzeug i s'ha descarregat el codi de referència (blockchain.tar.gz del repositori de la FIB). Cada node s'arrenca indicant el port amb -p.

```
python3 --version
python3 -m venv prbc
source prbc/bin/activate
pip3 install flask requests werkzeug
python3 blockchain.py -p 5000
```

2.2 Part I: transaccions i minat

La xarxa està formada per dos nodes, cadascun un servidor Flask: el node 1 al port 5000 i el node 2 al port 5001. Les operacions es fan amb curl sobre l'API web. Cada bloc té els camps index, previous_hash, proof, timestamp i la llista de transactions. Primer s'han escrit dues transaccions al node 1 i s'ha minat un bloc:

```
curl -X POST -H "Content-Type: application/json" -d '{"sender":"A","recipient":"B","amount":8,"order":1}'
http://localhost:5000/transactions/new
curl -X POST -H "Content-Type: application/json" -d '{"sender":"B","recipient":"C","amount":5,"order":2}'
http://localhost:5000/transactions/new
curl http://localhost:5000/mine
curl http://localhost:5000/chain
```



Figura 1. Node 1: després de minar el primer bloc la cadena té longitud 2

Les dues transaccions han quedat dins el bloc 2. Al costat hi ha una tercera transacció amb sender «0» i amount 1: és la recompensa pel minat, que va a l'identificador del node. El bloc gènesi té previous_hash «1» i proof 100.

El camp proof és el resultat del Proof-of-Work: el node prova valors fins que el hash SHA-256 que combina l'últim proof amb el nou compleix la condició de dificultat (a la implementació de referència, començar per quatre zeros). Els valors observats (34655, 91082, 43752...) són de l'ordre de $16^4 = 65.536$ intents, coherent amb quatre zeros hexadecimals.



Figura 2. Node 1: després de minar dos blocs més, la cadena té longitud 4 (gènesi + 3 blocs minats).

Cada bloc guarda a previous_hash el hash del bloc anterior, i això és el que encadena la cadena. Els blocs 3 i 4 només contenen la recompensa, perquè ja no hi havia transaccions pendents. L'enunciat parla de «tres blocs» al node 1 i «dos» al node 2: són els blocs minats; el camp length els compta amb el gènesi, per això surt 4 i 3.



Figura 3. Node 2: la seva pròpia cadena, amb longitud 3 (gènesi + 2 blocs minats).

El node 2 té una cadena diferent: el gènesi coincideix, però els blocs 2 i 3 tenen altres previous_hash i proof, i la recompensa va a un altre identificador (d6aff95c... en lloc de 98f13953...). Mentre els nodes no es coneguin, les cadenes divergeixen.

2.3 Part Ib: registre de nodes i consens

Cada node es registra l'altre amb POST /nodes/register. La resposta mostra la llista de nodes coneguts (total_nodes). Després, /nodes/resolve fa que cada node demani la cadena als seus veïns i adopti la més llarga que sigui vàlida, comprovant la cadena de hashes.



Figura 4. Registre mutu dels dos nodes.

Amb aquest criteri, el node 1 (4 blocs) manté la seva cadena i respon «Our chain is authoritative», mentre que el node 2 (3 blocs) la substitueix per la del node 1. Així les dues acaben amb la mateixa cadena.

```
curl -X POST -H "Content-Type: application/json" -d '{"nodes":"http://localhost:5001"}' http://localhost:5000/nodes/register
curl -X POST -H "Content-Type: application/json" -d '{"nodes":"http://localhost:5000"}' http://localhost:5001/nodes/register
curl http://localhost:5000/nodes/resolve
curl http://localhost:5001/nodes/resolve
```

Un cop provades les operacions una a una, s'han agrupat en un script (script.sh) que arrenca els dos nodes i executa tota la seqüència.

2.4 Noves funcions de l'API

S'han afegit tres endpoints a blockchain.py: /nodes/list (nodes registrats), /validate (comprova la cadena de hashes del node) i /nodes/manipulate (altera un bloc perquè la validació falli). La manipulació es fa sobre el bloc 2 i no sobre l'últim, perquè només així el previous_hash del bloc següent deixa de coincidir.

```
@app.route('/nodes/list', methods=['GET'])
def list_nodes():
    return jsonify({
        'nodes': list(blockchain.nodes),
        'total_nodes': len(blockchain.nodes)
    }), 200

@app.route('/validate', methods=['GET'])
def validate():
    return jsonify({
        'valid': blockchain.valid_chain(blockchain.chain),
        'length': len(blockchain.chain)
    }), 200

@app.route('/nodes/manipulate', methods=['POST'])
def manipulate():
    if len(blockchain.chain) < 3:
        return jsonify({'message': 'Need at least 3 blocks'}), 400

    blockchain.chain[1]['transactions'] = [{
        'sender': 'FIB', 'recipient': 'FIB', 'amount': 100000000000000000000, 'order': 1
    }]
    return jsonify({'message': 'Chain manipulated (block 1 altered)'}), 200
```

Per provar-les s'ha preparat el node 2 amb una transacció i tres blocs minats, i s'han registrat els dos nodes.

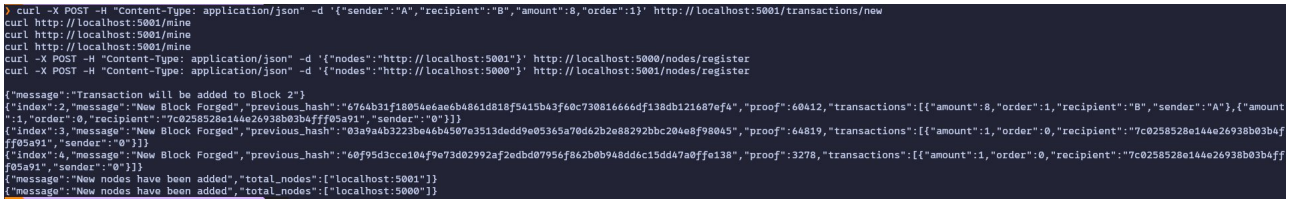


Figura 5. Preparació de les proves: transacció, tres blocs minats al node 2 i registre dels nodes.



Figura 6. Node 2: /nodes/list, /validate (true), /nodes/manipulate i /validate (false).

/nodes/list retorna el node 1 com a únic node registrat. Abans de manipular, la cadena és vàlida (valid: true, length: 4); després de /nodes/manipulate, el mateix node respon valid: false.



Figura 7. Cadena manipulada del node 2 (bloc 2 amb HACKER → HACKER de 1.000.000) i /nodes/resolve del node 1.

El bloc 2 conté ara una transacció fraudulenta, però el bloc 3 conserva el previous_hash original (03a9a4b3...), que ja no coincideix amb el hash del bloc 2 modificat: això és el que detecta /validate. Qualsevol canvi en un bloc trenca l'enllaç amb tots els següents.

En executar /nodes/resolve al node 1 (que només tenia el bloc gènesi després de reiniciar-se), la cadena del node 2 és més llarga (4 blocs) però invàlida, de manera que es rebutja i el node 1 respon «Our chain is authoritative». Així es veu que el consens només accepta cadenes vàlides, no només llargues.

2.5 Preguntes addicionals

1) Alternativa al Proof-of-Work. Opcions considerades:

- **Proof of Stake (PoS):** el creador del bloc s'escull amb probabilitat proporcional al saldo que té en joc, en lloc de gastar CPU.
- **Proof of Authority (PoA):** només uns nodes autoritzats i identificats poden signar blocs.

La més fàcil d'integrar al nostre codi és una versió simplificada de PoS: a /mine, en lloc de buscar un proof, el node tria un validador segons el saldo i el bloc el signa, i valid_chain comprova aquesta signatura en lloc de valid_proof. Cal tenir saldos i una distribució inicial al bloc gènesi, perquè ara només la recompensa de minat crea saldo i A acaba en negatiu.

2) Protecció de la creació de transaccions. Ara mateix qualsevol pot fer un POST a /transactions/new amb el sender que vulgui (a les proves s'ha enviat «A» sense cap prova d'identitat). Es pot protegir amb:

- **Signatures digitals (ECDSA/RSA):** l'emissor signa la transacció amb la clau privada i els nodes la verifiquen amb la pública, que alhora identifica el sender.
- **Autenticació de l'API** (API keys, JWT) i **HTTPS/TLS** per al canal.
- **Limitació de peticions** (rate limiting) per evitar abusos.

3) Estat a la blockchain. No: la cadena només guarda la llista de transaccions (més la recompensa de minat), no els saldos de A i B (figures 1 i 2). Per afegir-lo es poden calcular els saldos recorrent la cadena.

Un disseny més complet guardaria a cada bloc un hash de l'estat (com fa Ethereum) perquè els nodes puguin comprovar-lo.

4) Privacitat. No: qualsevol que consulti /chain veu emissor, receptor i import de totes les transaccions . L'obertura de les dades és el que permet validar, perquè cada node recalcula els hashes a partir del contingut dels blocs.

Hi ha tecniques que podrien ajudar a millorar la seguretat:

- **Zero-knowledge proofs (zk-SNARKs):** demostren que una transacció és vàlida sense revelar-ne les dades (Zcash).
- **Commitments i hashes a la cadena:** només es guarda el hash de la transacció (amb sal) i les dades van fora de la cadena.
- **Xifrat homomòrfic i canals privats** (Hyperledger Fabric), on només els participants veuen les dades.

2.6 Valoració

La pràctica permet veure de manera molt directa com funciona una blockchain: els blocs s'encadenen amb hashes, el Proof-of-Work fa costós crear-los, el consens tria la cadena més llarga i vàlida, i qualsevol manipulació queda detectada per la validació. Les proves amb curl fan que cada pas es pugui observar.

Com a limitacions, els dos nodes s'executen a la mateixa màquina, no hi ha cap autenticació de les transaccions, no es guarda cap estat de saldos, les dades són totalment públiques i el PoW és molt lleuger. En un cas real caldrien signatures, validació de saldos, més nodes i un consens més robust.